

8.16 模拟面试

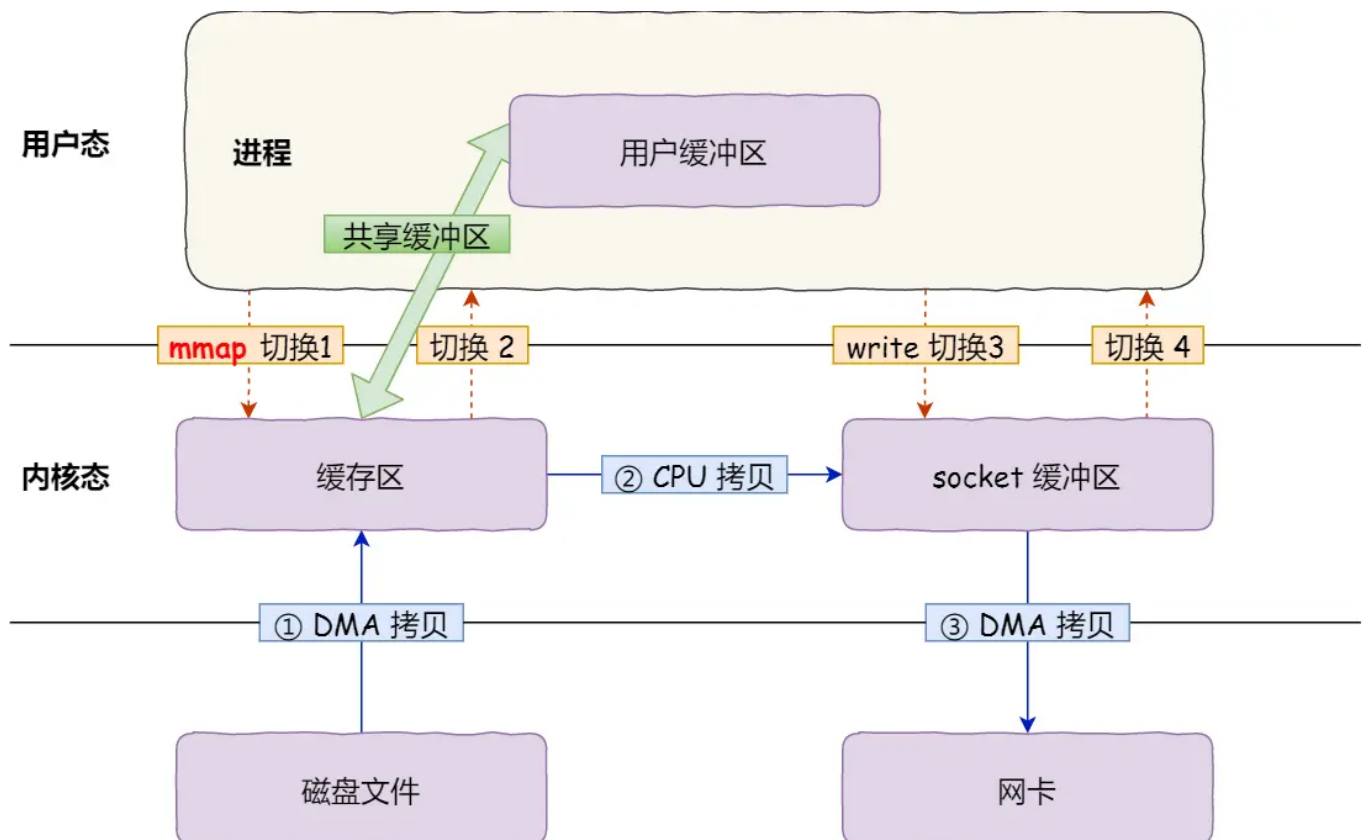
如何实现零拷贝

mmap+write

`read()` 系统调用的过程中会把内核缓冲区的数据拷贝到用户的缓冲区里，于是为了减少这一步开销，我们可以用 `mmap()` 替换 `read()` 系统调用函数。

```
1 buf = mmap(file, len);
2 write(socket, buf, len);
```

`mmap()` 系统调用函数会直接把内核缓冲区里的数据「映射」到用户空间，那么就不用再进行拷贝



应用进程与OS共享内核缓冲区中的数据，在应用程序调用 `write()`，操作系统直接将内核缓冲区的数据拷贝到 **socket 缓冲区**中，这一切都发生在内核态，由

CPU 来搬运数据；

减少了一次数据拷贝，依旧还有3次数据拷贝，并不是理想的零拷贝；

仍然需要 4 次上下文切换，因为系统调用还是 2 次。

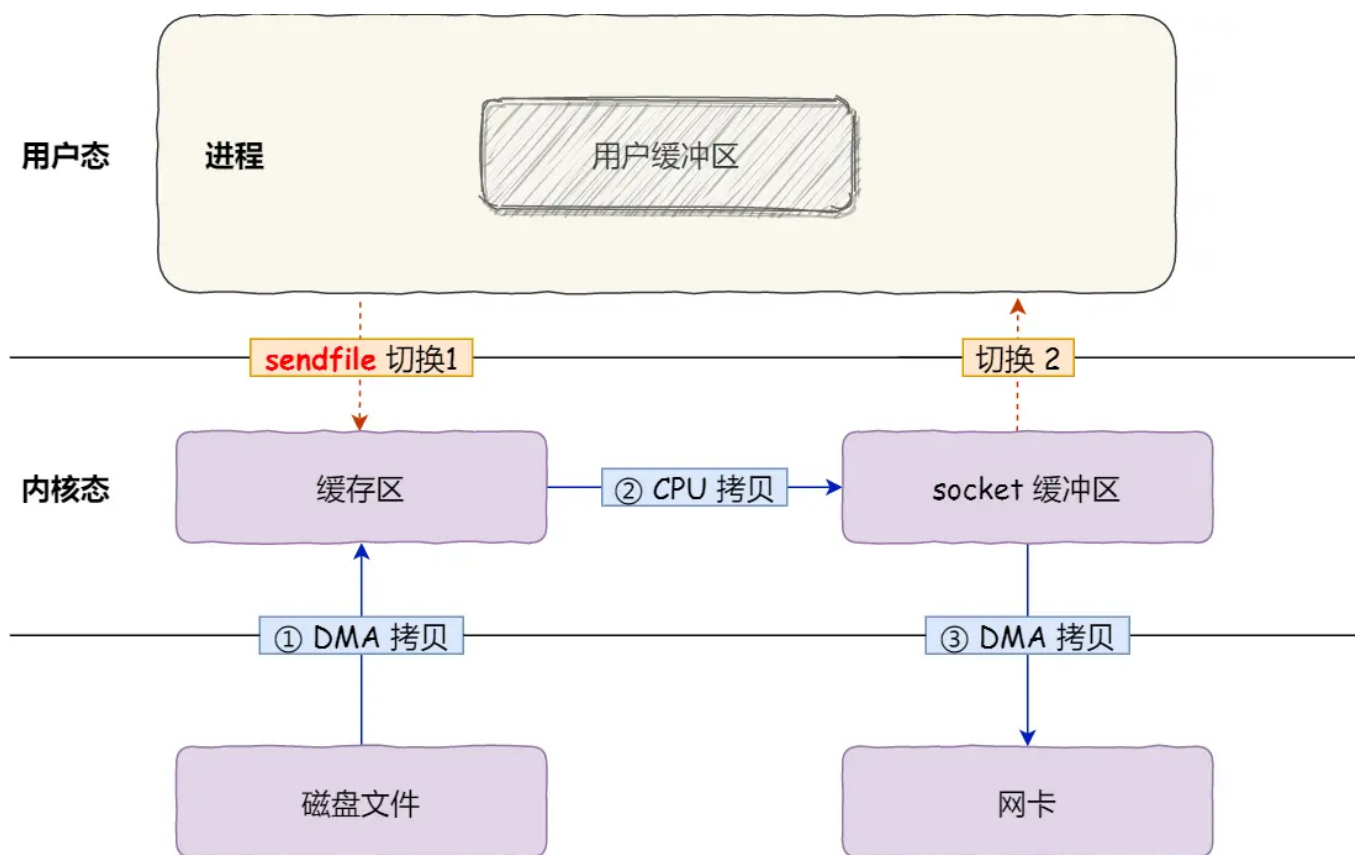
sendfile+SG-DMA

```
1 #include <sys/socket.h>
2 ssize_t sendfile(int out_fd, int in_fd, off_t* offset, size_t
  count);
```

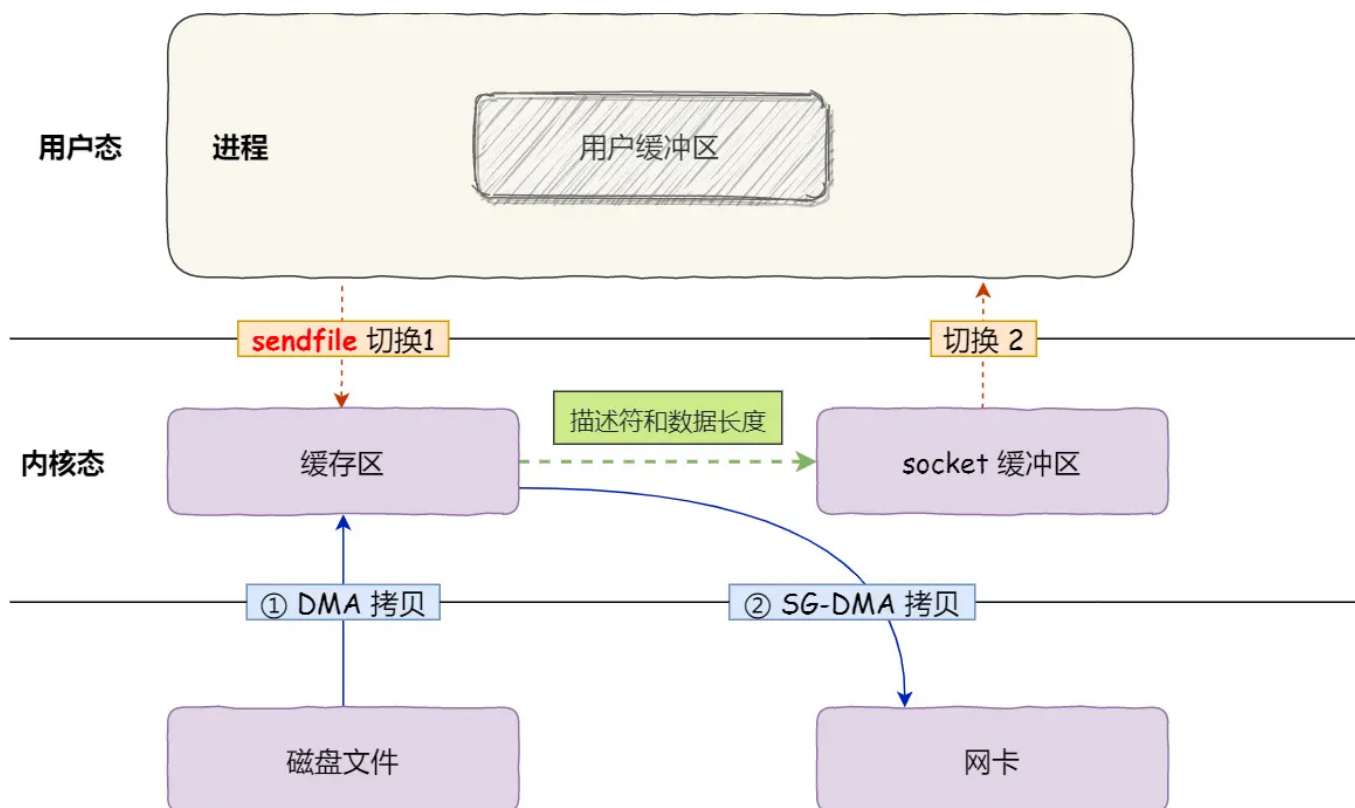
- out_fd: **写出文件描述符**，必须是**socket** (TCP 套接字)；旧版可以是任意文件
- in_fd: **读入文件描述符**，必须是**普通文件** (磁盘文件)；
- offset: 源端的偏移量
- count: 复制的数据长度；
- 返回实际复制数据的长度

可以替代前面的 `read()` 和 `write()` 这两个系统调用，减少了一次系统调用，也就减少了 2 次上下文切换的开销。

可以直接把内核缓冲区里的数据拷贝到 socket 缓冲区里，不再拷贝到用户态，这样就只有 2 次上下文切换，和 3 次数据拷贝。



但还没有实现真正的零拷贝，如果网卡支持SG-DMA (*The Scatter-Gather Direct Memory Access*) 技术 (与普通DMA不同) 还可以减少 内核缓冲到socket缓冲区的CPU拷贝;



- 首先通过 DMA 将磁盘上的数据拷贝到内核缓冲区里；
- 缓冲区描述符和数据长度传到 socket 缓冲区，样网卡的 SG-DMA 控制器就可以直接将内核缓存中的数据拷贝到网卡的缓冲区里；

这样就实现了真正的零拷贝（Zero-copy），**因为我们没有在内存层面去拷贝数据，也就是说全程没有通过CPU 来搬运数据，所有的数据都是通过 DMA 来进行传输的。

相较于传统的文件传输 只需要2次上下文切换和2次拷贝，还是DMA实现的；

DMA拷贝不算到拷贝次数中，因为任何一个读数据都会有这个操作（预读数据）

如何查看网卡是否支持 scatter-gather 特性：

```
1 $ ethtool -k eth0 | grep scatter-gather
2 scatter-gather: on
```

PageCache

零拷贝技术是基于 PageCache

在文件传输过程中第一次拷贝都是将 磁盘文件DMA拷贝到内核缓冲区中，这个内核缓冲区实际就是 PageCache,即**磁盘高速缓存**

优点：

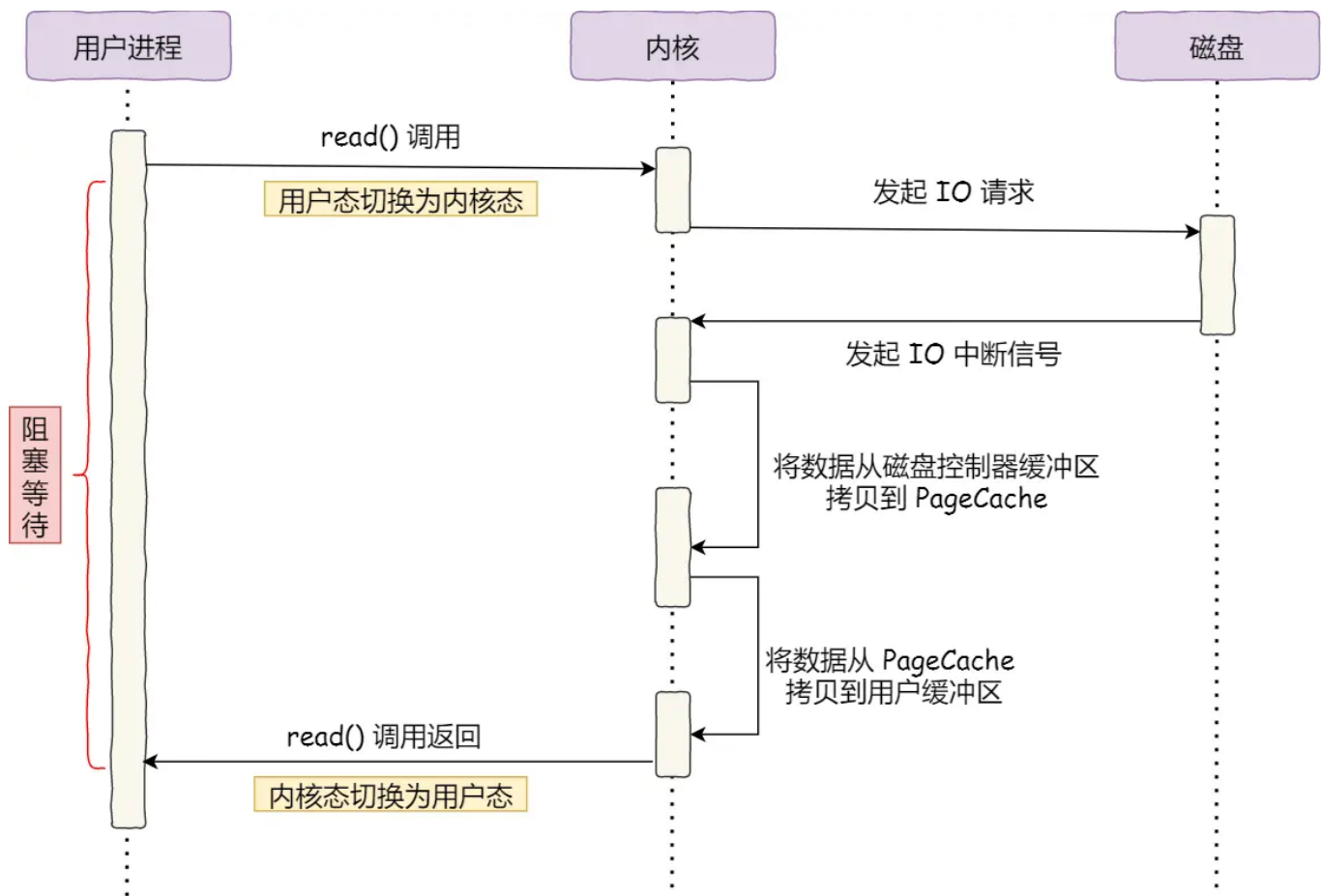
- 缓存最近被访问的磁盘中数据
- 预读磁盘数据功能；

但是在文件传输中 大文件会很快占满 PageCache,但实际其中没有多少热点数据，但是其他热点小文件无法用到，没有多少IO请求命中PageCache中的数据，那么就是白白**浪费了这个空间还消耗了DMA拷贝**；

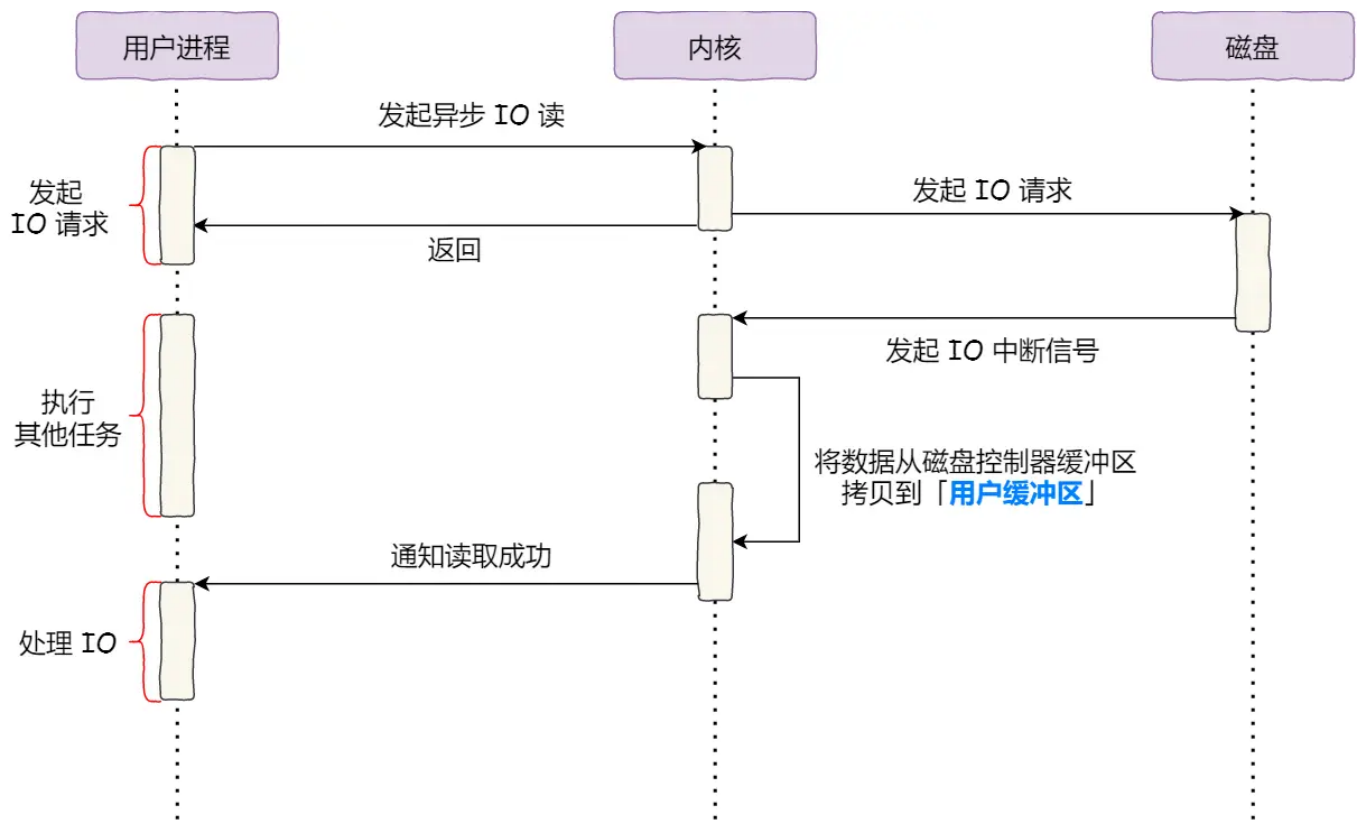
更实际一点说大文件传输就不适合用零拷贝技术；

大文件如何传输？

传统文件传输：



- 调用 `read` 方法时，**会阻塞着**，此时内核会向磁盘发起 I/O 请求，磁盘收到请求后，便会寻址，当磁盘数据准备好后，就会向内核发起 I/O 中断，告知内核磁盘数据已经准备好；
- 内核收到 I/O 中断后，就将数据从磁盘控制器缓冲区拷贝到 PageCache 里
- 最后，内核再把 PageCache 中的数据拷贝到用户缓冲区，于是 `read` 调用就正常返回了。



读操作分为两部分：

- 第一部分：**异步IO** 内核向磁盘发起读请求，但是可以**不等待数据就位就可以返回**，于是进程此时可以处理其他
- 第二部分：**直接IO** 内核将磁盘中的数据拷贝到用户缓冲区后，进程将接收到内核的**通知**，再去处理数据；

异步 I/O 并没有涉及到 PageCache，所以使用异步 I/O 就意味着要绕开 PageCache。

绕开 PageCache 的 I/O 叫直接 I/O，使用 PageCache 的 I/O 则叫缓存 I/O；

通常，对于磁盘，异步 I/O 只支持直接 I/O。

总结 传输文件的时候，我们要根据文件的大小来使用不同的方式：

- 传输大文件的时候，使用「异步 I/O + 直接 I/O」；
- 传输小文件的时候，则使用「零拷贝技术」；

注意：零拷贝技术是不允许进程对文件内容作进一步的加工的，比如压缩数据再发送。

Nginx优化——零拷贝技术

一般在文件传输中使用

但是不能一味地使用零拷贝，传输文件的时候，我们要根据文件的大小来使用不同的方式

- 传输大文件的时候，使用「异步 I/O + 直接 I/O」；
- 传输小文件的时候，则使用「零拷贝技术」；

```
1  location /video/ {  
2      sendfile on; //零拷贝  
3      aio on; //异步 IO  
4      directio 1024m; //直接 IO  
5  }
```

当文件大小大于 `directio` 值后，使用「异步 I/O + 直接 I/O」，否则使用「零拷贝技术」